MCS4302 Week 4 Lab – Measurement Sheet

Index number: 24440191

Repository URL: https://github.com/Deshan-Ruhunage/mcs4302-running-system

Seeds used: 42, 43, 44 (three runs, 50 sequential logical requests per condition to /flaky)

Retry experiment (average of 3 runs)

Condition	Success %	Mean ms	Mean of run p95 (ms)	HTTP calls
A: 0.3, no retry	66.7	58.3	86.7	50.0
B: 0.3, retry	98.7	217.3	880.0	73.7
C: 0.6, no retry	40.0	56.0	82.7	50.0
D: 0.6, retry	90.7	571.3	1721.0	111.0

Theoretical success: A 70%, B 99.19%, C 40%, D 87.04%.

Per-run results

Cond.	Seed	Success %	Mean ms	p95 ms	HTTP calls
A	42	62	61	89	50
B	42	98	220	813	74
C	42	32	57	84	50
D	42	88	650	1841	115
A	43	76	58	86	50
B	43	98	187	403	71
C	43	50	57	82	50
D	43	92	420	1555	100
A	44	62	56	85	50
B	44	100	245	1424	76
C	44	38	54	82	50
D	44	92	644	1767	118

Idempotency test (seed 42, 10 creates, 50% failure after save)

Test	Requested	Created	Duplicates	Reported failed
Key OFF	10	19	9	0
Key ON	10	10	0	0

Interpretation

Measured success rates were close to theory (A 66.7% vs 70%, B 98.7% vs 99.19%, C 40.0% vs 40%, D 90.7% vs 87.04%); the small differences are expected from 50-request samples. Retries bought a large reliability gain at a cost: at f = 0.6 success rose from 40% to 90.7%, but HTTP calls rose from 50 to 111 (2.2×) and mean-of-run p95 latency rose from about 83 ms to 1721 ms because of exponential backoff waits, so simply adding more retries at f = 0.6 would raise load and tail latency further for diminishing gains. Without jitter, 10,000 phones that fail together would all retry at the same 200/400/800 ms instants, creating synchronized load spikes (a thundering herd) that could keep an already struggling server overloaded. The idempotency test showed why POST retries need a key: without it, retries after a lost response created 9 duplicate items; with a key generated once per logical create, there were 0 duplicates.